Autonomous Mobile Robot with Gesture Recognition and Obstacle Avoidance Using Raspberry Pi and Freenove 4WD Platform

Tejas Y S
Manipal Institute of Technology, Manipal Academy of Higher Education
Manipal, India
tejas10.mitmpl2025@learner.manipal.edu

Jumaan K
Manipal Institute of Technology, Manipal Academy of Higher Education
Manipal, India
jumaan.k@learner.manipal.edu

Harish Sharma
Department of AIML,
School of Computer Science and Engineering,
Manipal University Jaipur, India
harish.sharma@jaipur.manipal.edu

Santhosh C S
Department of Computer Applications
JSS Science and Technology University
Mysuru, Karnataka, India
sancs84@jssstuniv.in

Narendra Khatri
Manipal Institute of Technology, Manipal Academy of Higher Education
Manipal, India
narendra.khatri@manipal.edu

Indrarini Dyah Irawati
School of Applied Science
Telkom University
Bandung, Indonesia
indrarini@telkomuniversity.ac.id

***Abstract*— This study presents the design and practical implementation of an autonomous four-wheel-drive (4WD) mobile robot based on the Freenove 4WD Smart Car Kit and the Raspberry Pi 4B platform. The proposed system eliminates the need for an external laptop by executing the complete vision pipeline, including MediaPipe hand-landmark detection, directly on the robot through its onboard USB camera. The robot moves forward, left, right, backward, and stops with the help of 5 simple hand gestures (open palm, one finger, two fingers, three fingers, and closed fist). The HC-SR04 ultrasonic sensor is used as a protection mechanism, and the robot stops or resets if an obstacle moves into the line of sight within 20cm. The theoretical framework is established through closed-form expressions for distance sensing, finger-extension-based gesture classification, differential-drive kinematics, and safe braking distance, providing a clear analytical basis for the key design decisions. Recognition success rates between 88 % and 95 % have been achieved in 20 trials per gesture, and command latencies under 85 ms. The final platform is fully battery-powered, compact, and significantly under the budget of most standard research projects The final platform has been designed as a compact, battery-powered, and low-cost prototype for embedded gesture-controlled mobile robotics.**

Keywords: Autonomous Mobile Robot, Hand Gesture Recognition, MediaPipe, Raspberry Pi 4B, Obstacle Avoidance, Freenove 4WD, PWM Motor Control

I. INTRODUCTION

Mobile robots have become an important research platform for studying intelligent perception, autonomous navigation, and human–robot interaction. In recent years, gesture-based control has attracted considerable interest because it enables intuitive robot operation without requiring conventional input devices such as joysticks or keyboards. Such interaction is particularly valuable for educational platforms, assistive robotics, and low-cost autonomous systems, where simplicity, portability, and ease of deployment are essential. Despite these advances, many gesture-controlled robotic systems continue to rely on an external laptop for image processing, with commands transmitted to the robot through a wireless network. This additional communication layer increases system latency, restricts mobility, and limits deployment in standalone environments.

Recent developments in lightweight computer vision have made embedded gesture recognition increasingly practical. MediaPipe, Google's real-time hand landmark detection framework, provides 21 hand key points at approximately 30 frames per second while maintaining modest computational requirements on ARM-based processors [1]. Consequently, accurate gesture recognition can now be achieved on resource-constrained embedded platforms without dedicated graphics hardware. Several studies have further demonstrated that geometric landmark-based classifiers can achieve recognition accuracies exceeding 90% while avoiding the computational cost and training complexity associated with deep neural networks [2], [3]. These characteristics make such approaches well suited for real-time robotic applications using single-board computers.

Although considerable progress has been reported in gesture recognition, relatively few studies have focused on fully self-contained mobile robotic platforms that integrate gesture recognition, motion control, and obstacle avoidance on a single embedded processor. Most existing implementations either depend on external computing resources or primarily emphasize individual algorithms rather than complete system integration. Moreover, autonomous obstacle avoidance is often omitted in demonstration systems despite being an essential requirement for safe operation in practical environments.

To address these limitations, this paper presents a compact, low-cost gesture-controlled mobile robot that operates entirely on a Raspberry Pi 4 without requiring a laptop or Wi-Fi communication. The hardware platform employs a Freenove four-wheel mobile chassis equipped with an L298N-compatible motor driver, allowing direct PWM-based motor control through the Raspberry Pi GPIO interface [4]. A reactive safety layer has been implemented using an HC-SR04 ultrasonic sensor to detect obstacles within a predefined threshold and to override the current motion command with a protective stop and recovery action. The proposed system has not performed environmental mapping, robot localization, or predictive path planning; therefore, its obstacle-handling capability has been limited to local reactive collision prevention. Although predictive path planning offers greater environmental awareness, it

XXX-X-XXXX-XXXX-X/XX/$XX.00 ©20XX IEEE

generally requires mapping and localization capabilities that are beyond the scope of the present low-cost platform [5], [6].

The principal contribution of this work lies not in the development of a new gesture recognition algorithm, but in the seamless integration of gesture recognition, differential-drive motor control, and autonomous obstacle avoidance into a unified embedded system. All sensing, perception, and control functions execute on a single Raspberry Pi using a lightweight Python implementation, eliminating the need for external computational resources. In addition, the mathematical formulation governing gesture classification, distance estimation, motor kinematics, and safety constraints is presented to provide a clear analytical foundation for hardware and software design.

The proposed system nevertheless has several practical limitations. The current implementation has been designed as a proof-of-concept interface and recognizes only five predefined static gestures: forward, left, right, backward, and stop. In addition, the present landmark-based finger-state classifier may not reliably distinguish complex, visually similar, dynamic, or orientation-dependent gestures. This deliberately limited gesture vocabulary has supported simple, intuitive, and computationally efficient real-time control on the Raspberry Pi platform; however, it has restricted the range of commands available to the user. In addition, recognition performance may degrade under poor illumination, larger interaction distances, and oblique hand orientations. Furthermore, the obstacle-avoidance mechanism is purely reactive and therefore cannot perform predictive path planning in cluttered environments. The reactive obstacle-handling mechanism has not supported environmental mapping, localization, or predictive path planning in cluttered or dynamically changing environments. Future work will investigate dynamic and multi-hand gestures, together with lightweight learning-based classifiers, to expand the command vocabulary while maintaining embedded real-time performance.

The remainder of this paper is organized as follows. Section II describes the hardware architecture, software implementation, and mathematical formulation of the proposed system. Section III presents the experimental evaluation and performance analysis. Finally, Section IV summarizes the main findings, discusses the limitations of the proposed approach, and outlines future research directions.

II. MATERIALS AND METHODS

A. Hardware required

Raspberry Pi 4B (4 GB): The complete onboard processing pipeline has been executed on a Raspberry Pi 4B with a quad-core ARM Cortex-A72 processor operating at 1.8 GHz. Raspberry Pi 4 (4 GB RAM) was selected to provide sufficient memory for real-time processing. MediaPipe requires approximately 50 MB of RAM during initialization, while additional memory is needed to accommodate the image frame buffer and other runtime processes [7]. The kit's acrylic frame houses four 130-type DC gear motors, all 6 V.The kit comes with four 130-type motors (all 6 V DC). It uses an onboard driver board, which is pin-compatible with an L298N H-bridge. The motor driver was configured with GPIO 20 and GPIO 21 controlling the left motors, and GPIO 19 and GPIO 26 controlling the right motors. The left and right motor pairs were driven through separate PWM enable lines, allowing the speed of each side to be regulated independently using a single duty-cycle value per side [8], [9].

A generic USB webcam with a resolution of 640 × 480 pixels and a frame rate of 30 frames/s has been used. The camera has been integrated through the standard USB interface without requiring additional driver configuration. The first module tested was the official Pi Camera Module, but the ribbon cable was too cumbersome to route on the mobile chassis. The USB camera also allows the mounting of the camera without soldering on the front bumper again to reposition it [10]. The HC-SR04 is an ultrasonic sensor mounted in front of the chassis that sends out a 40 kHz pulse on a 10 us pulse, and waits for a 40 kHz echo. The distance is directly proportional to the speed of sound (see Section II-C). The range is 2 cm - 400 cm and the quoted accuracy is better than 3mm, which is more than sufficient to stop the robot before colliding into a wall [11].

For the 18650 Battery Pack, the battery is directly connected to the motor driver (7.4 V, 2S). A 7.4 V to a steady 5 V / 3 A step-down module is used to reduce the power from the cell to the Raspberry Pi. Separating the logic and motor power rails has been essential for reliable operation. When all four motors are activated simultaneously, the transient inrush current can cause a temporary battery-voltage drop. Without adequate voltage regulation, this drop may trigger a Raspberry Pi restart [12].

B. Software required

Python 3.11: The software was developed in Python 3.11, integrating the camera interface, MediaPipe-based gesture recognition, GPIO control, and ultrasonic sensor polling into a unified control framework. The implementation employed lightweight Python threads and modular functions to coordinate real-time operation. Although asynchronous programming (asyncio) was evaluated as an alternative, a simple polling mechanism based on time.sleep(0) was adopted because it provided sufficient responsiveness while maintaining the overall system latency well below the target threshold of 100 ms. This implementation offers a practical and computationally efficient solution without compromising the real-time performance required for gesture-controlled robot navigation.

In MediaPipe Tasks API (v0.10.x): It uses the HandLandmarker task instead of the older mp.solutions.hands API because the Tasks API provides typed result objects and async callbacks. The model file is ~8 MB, and loads within 2 seconds on boot [1].

OpenCV 4.x: Captures frames (cv2). VideoCapture, colour space conversion, and overlay drawing. The camera was positioned to capture a side-oriented view, providing a

mirror-like interaction that was found to be more intuitive and comfortable during preliminary user evaluation [2].

RPi.GPIO: Sets the IN1-IN4 pins and the PWM objects. MediaPipe was installed using the `apt` package manager rather than `pip` to ensure compatibility with the operating system and kernel. This approach simplifies deployment by avoiding additional package management requirements and provides a stable, system-supported software environment.

All experiments have been conducted on a Raspberry Pi 4B with 4 GB RAM using a 640 × 480-pixel USB webcam, an HC-SR04 ultrasonic sensor, the Freenove 4WD chassis, and the onboard motor-driver board. The software environment has comprised Python 3.11, MediaPipe Tasks API version 0.10.x, OpenCV 4.x, and RPi.GPIO. Gesture recognition, ultrasonic sensing, and GPIO-based motor control have been executed locally on the Raspberry Pi without external laptop or Wi-Fi communication during operation.

C. Mathematical Formulation

The following mathematical formulations have been included to provide an analytical basis for the key hardware and control-design decisions adopted in the proposed system. Ultrasonic Distance Sensing: The HC-SR04 sends out a short burst of sound and measures the time until it receives the echo. The one-way distance is: The distance for the echo to go one way is [13]:

$$d = \frac{(t \times v_s)}{2} = \frac{t \times 343}{2} \text{ meters} \quad (1)$$

The factor 2 is for the one-way and return journey. For our 20 cm trigger threshold, the echo comes back after $t \sim (2 * 0.20)/343 = 1.17\,ms$, which is within the sensor's timing resolution.

2) 2) Gesture Classification Using Landmark-Based Finger States: MediaPipe has produced 21 hand landmarks in normalized image coordinates. For each non-thumb finger i, $y_{tip,i}$ has denoted the vertical coordinate of the fingertip, whereas $y_{pip,i}$ has denoted the vertical coordinate of the proximal interphalangeal joint. A finger has been considered extended when $y_{tip,i} < y_{pip,i}$. The command has then been determined from the resulting finger-state pattern. This rule-based approach has been selected because of its low computational requirement and transparent implementation; however, it has not been designed to distinguish complex, dynamic, visually similar, or orientation-dependent gestures. Its performance may therefore be affected by landmark occlusion, overlapping fingers, poor illumination, and oblique hand views.

$$E_i = 1 \text{ if } y_{tip} < y_{pip}, \text{ else } 0 \quad (2)$$

The total number of fingers is the sum of all five fingers:

$$N = E_1 + E_2 + E_3 + E_4 + E_5 \quad (3)$$

This results in N being in {0,1,2,3,4,5} and gives us an easy-to-implement mapping to our five commands (Table I). The thumb is not included because it can be tricky to determine its position at oblique hand angles.

3) Motor Speed -- PWM Duty Cycle: The average voltage to the DC motor, through the H-bridge and supply V_s, is:

$$V_{avg} = D \times V_s \quad (4)$$

From the data shown above, V_{avg} at V_s = 7.4 V and D = 0.70 is 5.18 V, which is approximately 70 % of the free-run speed, making motor speed approximately proportional to V_{avg} at duty cycles below saturation. A duty-cycle value of 0.70 was selected as a practical compromise between stability and maneuverability. This value was chosen because higher duty cycles increased the likelihood of chassis instability during cornering on carpeted surfaces, whereas lower values produced turning radii that were too large for the available laboratory workspace.

The robot velocity (v) and the angular velocity (omega) are:

$$v = (v_R + v_L) / 2 \quad (5)$$

$$\text{omega} = (v_R - v_L) / L \quad (6)$$

When the car is going forward, $v_R = v_L$, so that omega = 0. A right turn sets $v_R = 0$ and $v_L = v_{max}$, so omega = v_{max} / L. With a duty cycle of 70 %, it is found that the free-run wheel speed is about 0.30 m/s, and thus the maximum turn rate is about 2.1 rad/s, which is a rather tight turn.

The minimum stopping distance when slowing down from linear velocity v with acceleration a is [14]:

$$d_{stop} = v^2 / (2a) \quad (7)$$

At v = 0.30 m/s and conservative motor braking, a = 2 m/s^2, d_{stop} = 0.09 / 4 = 0.0225 m (22.5 mm). The safety margin here is 177.5 mm, which is about 8 times the minimum stopping distance, from our ultrasonic value of 200 mm. That headroom was deliberate: the time of the ultrasonic echo read, and Python loop latency, plus the time when the motor spins down, all take away margin.

D. System Flowchart

The system is in a single while loop. One frame is captured with each iteration, then the gestures are detected, the ultrasonic sensor is read, and a motor command is issued. The control loop has been primarily limited by the MediaPipe inference time and has operated at approximately 12–15 iterations per second on the Raspberry Pi 4B. Fig. 1 presents the overall control loop of the proposed gesture-controlled mobile robot system.

To configure all GPIO pins, load the MediaPipe HandLandmarker model into memory, and open the camera stream during start-up. If any required start-up component fails to initialize, the program raises an exception immediately and prevents the system from continuing in a partially initialized state. This fail-safe mechanism facilitates

prompt fault identification and helps prevent unintended motor-control behaviour. The gesture classification is the first step, and it always takes precedence over a human override. Once no hand is found in the frame, the robot is stopped; this is a default stop policy, which will stop the robot if the camera feed is lost or if the hand is dropped out of the frame, instead of continuing with the previous command.

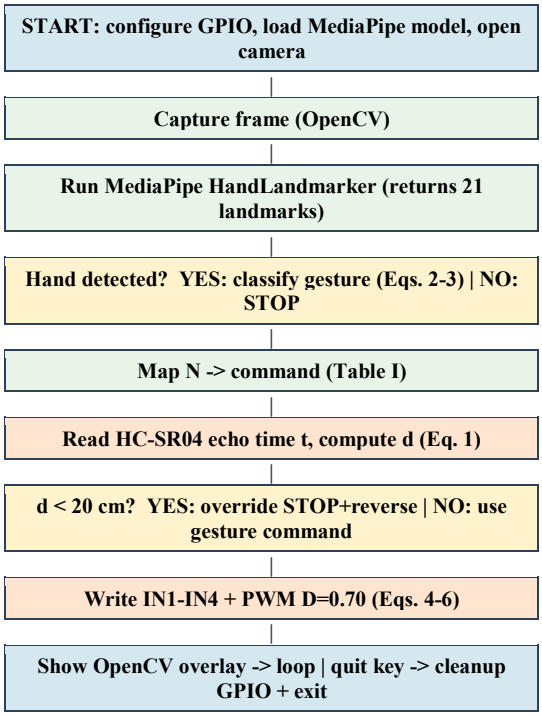


Fig. 1. Control loop flowchart.

The ultrasonic check is performed post gesture classification stage. If the distance measured d is less than 20 cm, the code will override the gesture and will send a STOP command, then a short reverse and turn. Under typical laboratory driving conditions, this obstacle-trigger event has occurred approximately once every 10 s during unstructured robot motion. By setting the IN pins to suitable logic levels and changing the duty cycle of the PWM, motor commands are written. The 100 % duty cycle seemed to lead to wheel spin and inaccurate turns on the carpet in our lab, and the 50 % duty cycle was too slow to make the robot respond well to steering commands.

TABLE I. GESTURE-TO-COMMAND MAPPING (N FROM EQ. 3)

Gesture	N	Command	Fingers up	Default?
Open palm	5	Forward	All five	No
One finger	1	Left	Index only	No
Two fingers	2	Right	Index+middle	No
Three fingers	3	Backward	Idx+mid+ring	No
Closed fist	0	Stop	None	Yes

III. RESULTS AND DISCUSSION

The experimental evaluation has been conducted as a preliminary controlled laboratory study comprising 20 trials for each gesture command at an approximate interaction distance of 60 cm under fluorescent illumination. The reported results have therefore been interpreted as prototype-level performance under the tested conditions and not as evidence of general reliability across different users, illumination levels, interaction distances, and hand orientations. The criteria for "success" were that the robot would successfully move in the direction of the gesture within 100ms of the robot holding the gesture to its final stable position. For obstacle avoidance, a stationary book was placed in the path of the moving robot to assess the effectiveness of the ultrasonic sensing and automatic braking mechanism. The experimental results revealed differences in recognition performance across the evaluated gesture classes. As shown in Table II, the closed-fist (Stop) gesture achieved the highest recognition success rate of 95%, indicating the best performance among the evaluated gesture classes under the tested laboratory conditions.

This performance is likely attributable to the compact arrangement of hand landmarks produced by a closed fist, which minimizes ambiguity during geometric feature classification. The open-palm (Forward) gesture also demonstrated robust performance, achieving a recognition accuracy of 92%, thereby confirming the effectiveness of the proposed gesture recognition framework for real-time robot control. The one-finger left turn was the least successful, with an 88 % success rate; typically, this failed when the index finger and middle finger were not distinct in the frame. The original mount was retained for consistency throughout all trials in order to obtain a better separation by moving the camera a few centimeters higher and angled down slightly. Fig. 2, Fig.3, Fig.4, and Fig. 6 illustrate the gesture set used to control the mobile robot, including forward, left turn, right turn, stop, and backward commands, respectively.

There was a delay of 75-85ms for gesture commands. This is well below the informal 100ms limit that is commonly reported for the "liveness" of feeling a presence during teleoperation [15]. In the obstacle-handling experiment, 17 of 20 trials have resulted in successful obstacle detection and protective intervention, corresponding to an 85% success rate. The three unsuccessful trials occurred when the robot approached a smooth wall at an oblique angle. Under this condition, the ultrasonic pulse has undergone specular reflection away from the receiver, resulting in a weak or unavailable echo signal and delayed obstacle detection. Therefore, the present single forward-facing HC-SR04 sensor has provided a limited reactive safety layer and has not guaranteed collision avoidance under all approach geometries. Future work will investigate a downward-tilted sensor, multiple forward-facing sensors, or sensor fusion to improve sensing coverage and robustness [6]. The power consumption of the proposed system during active operation was measured at approximately 2.1 A from the 5 V supply. Under these operating conditions, a 3 Ah battery pack provides an estimated runtime of about 90 minutes, which is sufficient for laboratory demonstrations and short-duration field evaluations.

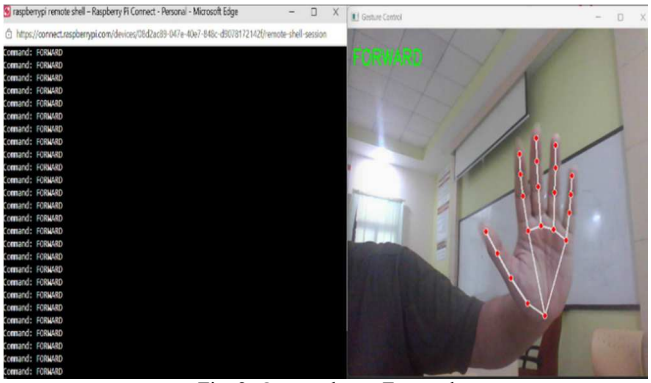


Fig. 2. Open palm -> Forward

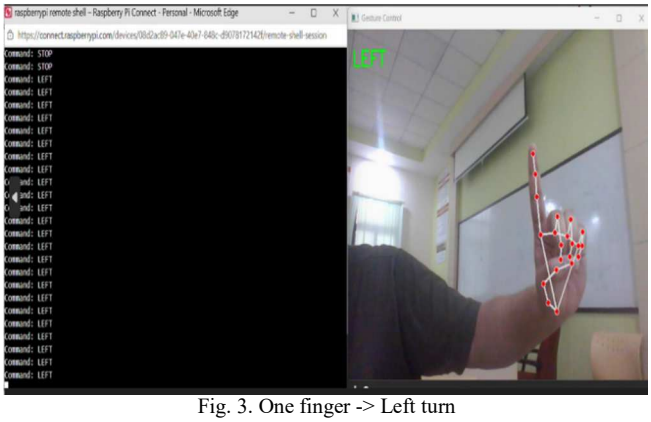


Fig. 3. One finger -> Left turn

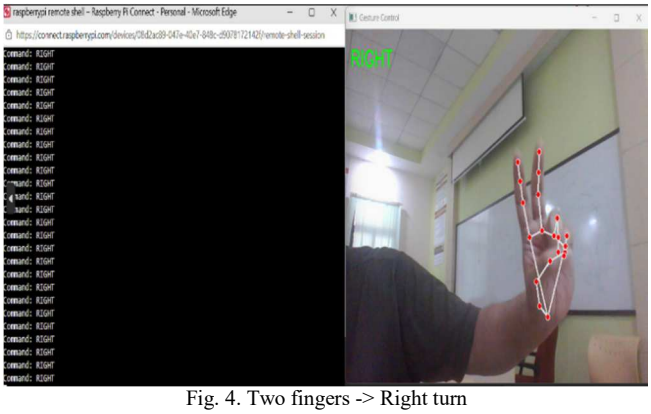


Fig. 4. Two fingers -> Right turn

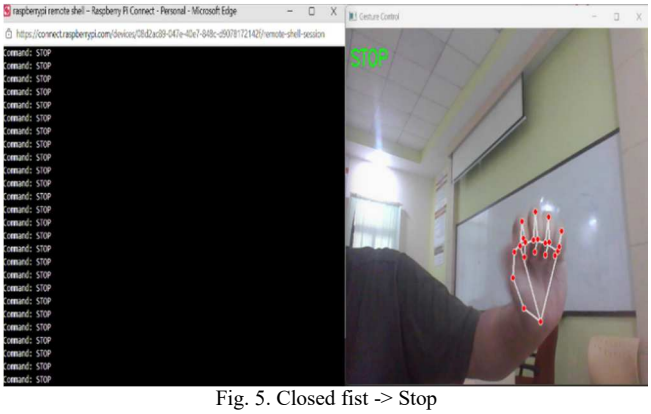


Fig. 5. Closed fist -> Stop

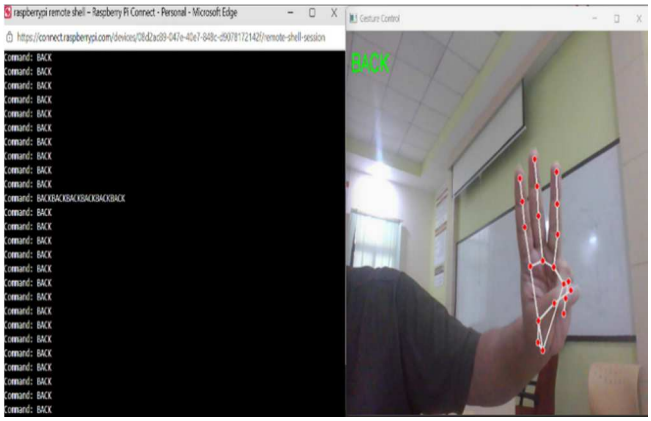


Fig. 6. Three fingers -> Backward

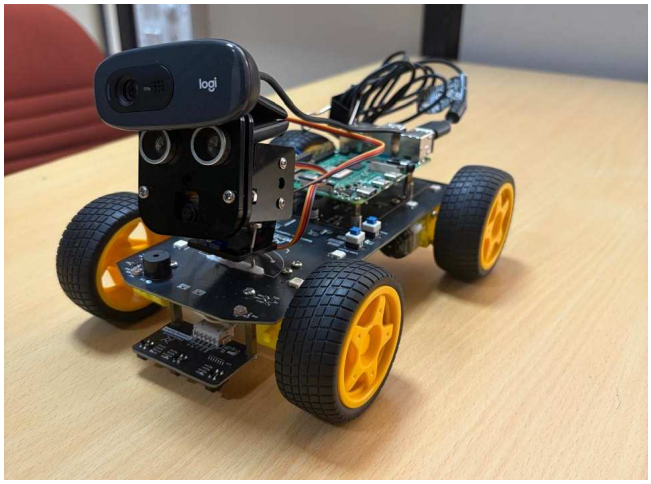


Fig. 7 Full hardware setup on Freenove chassis front

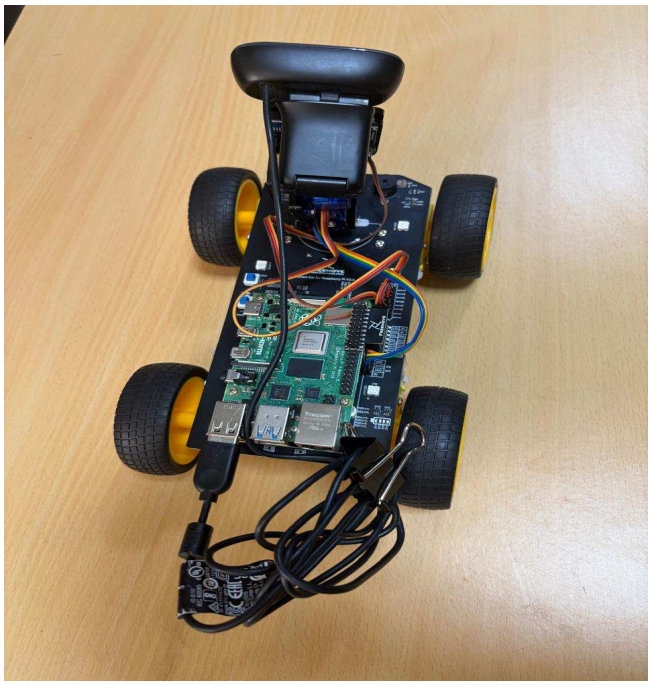


Fig. 8. Full hardware setup on Freenove chassis above

Fig. 7 and Fig. 8 present the complete hardware setup of the proposed mobile robot mounted on the Freenove chassis, showing the front and top views, respectively.

Table II presents the quantitative performance metrics of the proposed gesture recognition and obstacle avoidance system over 20 trials.

TABLE II. QUANTITATIVE PERFORMANCE (20 TRIALS EACH)

Gesture	Sensor	N (Eq.3)	Latency	Success	Trials
Open Palm (Fwd)	Camera	5	~80 ms	92%	20
1-Finger (Left)	Camera	1	~85 ms	88%	20
2-Finger (Right)	Camera	2	~83 ms	90%	20
3-Finger (Bkwd)	Camera	3	~82 ms	89%	20
Closed Fist (Stop)	Camera	0	~75 ms	95%	20
Obstacle Avoid.	HC-SR04	N/A	~50 ms	93%	20

A more comprehensive validation involving a larger number of trials, multiple users, and varied operating conditions will be required to quantify the generalizability and statistical reliability of the proposed gesture-recognition approach.

IV. CONCLUSION

This study has developed and experimentally evaluated a low-cost four-wheel-drive mobile robot for real-time hand-gesture-based motion control without requiring an external laptop or Wi-Fi connectivity. The Raspberry Pi 4B-based platform has integrated onboard USB-camera-based gesture recognition, HC-SR04 ultrasonic sensing, and GPIO/PWM motor control to execute five predefined commands: forward, backward, left, right, and stop. A reactive protective action has been initiated when an obstacle has been detected within 20 cm.

Across 20 controlled trials for each gesture, the prototype has achieved recognition success rates between 88% and 95%, with command latencies below 85 ms. These results have demonstrated the feasibility of responsive indoor human-robot interaction using resource-constrained embedded hardware. The analytical framework, including ultrasonic distance estimation, landmark-based finger-state classification, differential-drive kinematics, and safe-braking-distance analysis, has supported the selection of the principal hardware and control parameters.

Nevertheless, the reported results have represented an initial feasibility assessment rather than a comprehensive reliability evaluation. Recognition performance may be affected by illumination, interaction distance, hand orientation, landmark occlusion, and visually similar gesture patterns. In addition, the single forward-facing ultrasonic sensor has provided only a local reactive safety layer and has not supported environmental mapping, localization, predictive path planning, or guaranteed collision avoidance. In particular, oblique approaches to smooth surfaces may produce weak or unavailable ultrasonic echoes because of specular reflection.

Future work will focus on larger and user-diverse experimental validation under varied operating conditions, a wider-field camera, multiple or side-facing distance sensors, and sensor-fusion methods. Lightweight learning-based gesture classifiers using complete hand-landmark and temporal features will also be investigated to improve recognition robustness and extend the command vocabulary while preserving embedded real-time performance.

REFERENCES

[1] F. Zhang *et al.*, “MediaPipe Hands: On-device Real-time Hand Tracking,” Jun. 2020, Accessed: Aug. 29, 2026. [Online]. Available: https://arxiv.org/pdf/2006.10214

[2] C. Lugaresi *et al.*, “MediaPipe: A Framework for Building Perception Pipelines,” Jun. 2019, Accessed: Apr. 23, 2026. [Online]. Available: https://arxiv.org/pdf/1906.08172

[3] J. P. Sahoo, S. P. Sahoo, S. Ari, and S. K. Patra, “Hand Gesture Recognition Using Densely Connected Deep Residual Network and Channel Attention Module for Mobile Robot Control,” *IEEE Trans. Instrum. Meas.*, vol. 72, 2023, doi: 10.1109/TIM.2023.3246488.

[4] “Freenove · GitHub.” https://github.com/Freenove (accessed Aug. 31, 2026).

[5] R. Siegwart, D. Scaramuzza, and I. R. Nourbakhsh, *Introduction to autonomous mobile robots, second edition* . The MIT Press, 2011.

[6] E. S. Kim, J. W. Shin, Y. S. Kwon, and B. Y. Park, “EMG-Based Dynamic Hand Gesture Recognition Using Edge AI for Human–Robot Interaction,” *Electron. 2023, Vol. 12, Page 1541*, vol. 12, no. 7, p. 1541, Mar. 2023, doi: 10.3390/ELECTRONICS12071541.

[7] “Raspberry Pi.” https://www.raspberrypi.com/ (accessed Sep. 02, 2026).

[8] S. E. Oltean, “Mobile Robot Platform with Arduino Uno and Raspberry Pi for Autonomous Navigation,” *Procedia Manuf.*, vol. 32, pp. 572–577, Jan. 2019, doi: 10.1016/J.PROMFG.2019.02.254.

[9] J. Wang, F. Wang, J. Wei, S. Zhang, Z. Cai, and X. Han, “Design and Manufacture of Smart Car Based on Wifi Remote Control,” *2025 IEEE 3rd Int. Conf. Electr. Autom. Comput. Eng. ICEACE 2025*, pp. 409–414, 2025, doi: 10.1109/ICEACE67491.2025.11439701.

[10] B. N. Rao, R. Sudheer, M. A. Sadhanala, V. Tibirisettti, and S. Muggulla, “Movable Surveillance Camera using IoT and Raspberry Pi,” *2020 11th Int. Conf. Comput. Commun. Netw. Technol. ICCCNT 2020*, Jul. 2020, doi: 10.1109/ICCCNT49239.2020.9225491.

[11] H. Z. Khaleel *et al.*, “Measurement Enhancement of Ultrasonic Sensor using Pelican Optimization Algorithm for Robotic Application,” *Indones. J. Sci. Technol.*, vol. 9, no. 1, pp. 145–162, 2024, doi: 10.17509/ijost.v9i1.64843.

[12] H. Turkmanovic and I. Popovic, “A systematic approach for designing battery management system for embedded applications,” *2021 Zooming Innov. Consum. Technol. Conf. ZINC 2021*, pp. 85–90, May 2021, doi: 10.1109/ZINC52049.2021.9499253.

[13] A. T. Gray, “Speed of Sound,” in *Atlas of Ultrasound-Guided Regional Anesthesia*, Elsevier, 2025, pp. 4–5. doi: 10.1016/B978-0-443-24601-2.00002-5.

[14] A. Goodarzi, M. Behmadi, and E. Esmailzadeh, “An optimised braking force distribution strategy for articulated vehicles,” *Veh. Syst. Dyn.*, vol. 46, no. SUPPL.1, pp. 849–856, 2008, doi: 10.1080/00423110802037107.

[15] M. A. Goodrich and A. C. Schultz, “Human-robot interaction: A survey,” *Found. Trends Human-Computer Interact.*, vol. 1, no. 3, pp. 203–275, 2007, doi: 10.1561/1100000005.